

Practice Exam #2

SECTION I

Time — 90 minutes
Number of questions — 42
Percent of total grade — 55

1. Consider the following code segment.

```
int a = 1;  
int b = 2;  
double c = a/b;  
double d = (double) (a/b);  
System.out.println(c + d);
```

What is printed when the code segment is executed?

- (A) 0
 - (B) 0.0
 - (C) 0.5
 - (D) 1.0
2. Which of the following expressions will evaluate to `true` if and only if `x` and `y` are boolean variables with different values?
- (A) `(x || y) && (!x || !y)`
 - (B) `(x || y) && !(x && y)`
 - (C) `(x && !y) || (!x && y)`
 - (D) All of the above
3. Which of the following statements will result in a syntax error?
- (A) `String x = "123";`
 - (B) `Integer x = "123";`
 - (C) `String[] x = {"123"};`
 - (D) None of the above

4. Consider the following statement.

```
System.out.println("yes\\no".indexOf("\\no"));
```

What is printed when the statement is compiled and executed?

- (A) -1
 - (B) 3
 - (C) 4
 - (D) Nothing, due to a syntax error
5. What is the output when the following code segment is compiled and executed?

```
int[] arr = {-3, 0, 2, -5, 6, 4};

for (int i = 1; i < arr.length; i++)
{
    if (arr[i] > 0 && arr[0] < 0)
    {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
    }
}

for (int x : arr)
{
    System.out.print(x + " ");
}
```

- (A) 6 0 2 -5 -3 4
- (B) 4 0 2 -5 6 -3
- (C) 2 6 4 -3 0 -5
- (D) 2 0 -3 -5 6 4

6. Consider the following method.

```
/**
 * Returns the greatest common divisor of a and b.
 * Precondition: a > 0 and b > 0
 */
public static int gcd(int a, int b)
{
    int d = a;
    if (b < d)
    {
        d = b;
    }

    while (a % d != 0 || b % d != 0)
    {
        d--;
    }

    return d;
}
```

How many times will modulo division be performed when `gcd(12, 15)` is called?

- (A) 14
 - (B) 20
 - (C) 24
 - (D) 30
7. What is the result when the following code segment is compiled and executed?

```
Double x = 2.0;           // Line 1
double y = Math.pow(x, 3); // Line 2
Integer p = (int)(y + 0.5); // Line 3
System.out.println(p);
```

- (A) Incompatible types error on Line 1
- (B) Incompatible types error on Line 2
- (C) Incompatible types error on Line 3
- (D) 8 is displayed

8. What is the best description of the return value of the method `propertyX` below?

```
/**
 * Precondition: v.length >= 2
 */
public boolean propertyX(int[] v)
{
    boolean flag = false;

    for (int i = 0; i < v.length - 1; i++)
    {
        flag = flag || (v[i] == v[i+1]);
    }

    return flag;
}
```

- (A) Returns `true` if the elements of `v` are sorted in ascending order, `false` otherwise
 - (B) Returns `true` if the elements of `v` are sorted in descending order, `false` otherwise
 - (C) Returns `true` if `v` has two adjacent elements with the same value, `false` otherwise
 - (D) Returns `true` if `v` has two elements with the same value, `false` otherwise
9. The following method is intended to remove from `ArrayList<Integer> list` all elements whose value is less than zero.

```
public void removeNegatives(ArrayList<Integer> list)
{
    int i = 0, n = list.size();

    while (i < n)
    {
        if (list.get(i) < 0)
        {
            list.remove(i);
            n--;
        }
        i++;
    }
}
```

For which lists of `Integer` values does this method work as intended?

- (A) Only lists that do not contain negative values in consecutive positions
- (B) Only lists in which all the negative values occur before all the positive values
- (C) Only lists in which all the positive values occur before all the negative values
- (D) All lists

10. A class `Turtle` has a void method `forward(int d)` that moves the turtle forward by `d` units.

A class `TurtleRace` has a list of `Turtle` objects:

```
private ArrayList<Turtle> runners;
```

`TurtleRace`'s method `move` is supposed to move all the turtles. Kim programmed it with a for-each loop —

```
public void move(int d)
{
    for (Turtle t : runners)
    {
        t.forward((int) (d*Math.random()));
    }
}
```

— but Zane decided to replace that loop with a regular for loop:

```
public void move(int d)
{
    for (int i = 0; i < runners.size(); i++)
    {
        runners.get(i).forward((int) (d*Math.random()));
    }
}
```

What is a good reason for this change?

- (A) No good reason; Zane just wasn't sure how for-each loops work.
 - (B) The for loop with `get(i)` is more conventional programming style.
 - (C) The for-each loop didn't work because it cannot change objects in a list.
 - (D) The for loop with `get(i)` is more efficient.
11. What is the result when the following code segment is compiled and executed?

```
int m = 4, n = 5;
double d = Math.sqrt((m + n)/2);
System.out.println(d);
```

- (A) `ArithmeticException`
- (B) Syntax error "`sqrt(double)` in `java.lang.Math` cannot be applied to `int`"
- (C) 2.0 is printed
- (D) 2.1213203435596424 is printed

12. Consider the following method.

```
/**
 * Returns the number of times the digit d occurs in the
 * decimal representation of n.
 * Precondition: n and d are non-negative integers
 */
private int findDigit(int n, int d)
{
    int count = 0;

    < statement 1 >

    while (n > 0)
    {
        if (n % 10 == d)
        {
            count++;
        }
        < statement 2 >
    }

    return count;
}
```

Which of the following could replace < statement 1 > and < statement 2 > to make findDigit work as described in the comment for this method?

	<u>< statement 1 ></u>	<u>< statement 2 ></u>
(A)	if (n == 0) return 1;	n /= 10;
(B)	if (n == 0 && d == 0) count++;	n /= 10;
(C)	if (n == 0) return 1;	d *= 10;
(D)	if (d == 0) count++;	n -= n % 10;

13. Assume that a and b are properly declared and initialized boolean variables. Consider the following expression.

`!( !a || b) || (!a && b)`

When does the expression evaluate to true?

- (A) When a and b have different values
- (B) When a and b have the same value
- (C) When both a and b have the value true and only then
- (D) When both a and b have the value false and only then

14. Consider the following code segment.

```
String band = "ANAMANAGUCHI";  
  
System.out.println(band.substring(1, 4).  
                    compareTo(band.substring(5, 8)));
```

What is printed when the code segment is executed?

- (A) true
- (B) false
- (C) A negative integer
- (D) A positive integer

15. Consider the following code segment.

```
int num = 5;  
while (num >= 0)  
{  
    num -= 2;  
}  
System.out.print(num);
```

What is printed when the code segment is executed?

- (A) -1
- (B) -2
- (C) 0
- (D) 2

16. Consider the following method from `ClassX`.

```
private int modXY(int x, int y)  
{  
    r = x / y;  
    return x % y;  
}
```

If `ClassX` compiles with no errors, which of the following must be true?

- (A) `r` must have the type `double`.
- (B) `r` is not a local variable in the `modXY` method.
- (C) `r` must be a static variable in `ClassX`.
- (D) `r` must be a local variable in another method of `ClassX`.

17. Suppose `ArrayList<Integer> numbers` and `ArrayList<String> names` are created as follows:

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
Integer x = 1;
numbers.add(x);
numbers.add(x);

ArrayList<String> names = new ArrayList<String>();
names.add(0, "Anya");
names.add(0, "Ben");
names.add(0, "Cathy");
```

Consider the following code segment.

```
for (Integer i : numbers)
{
    names.remove(i);
}
for (String name : names)
{
    System.out.print(name + " ");
}
```

What is the result when the code segment is executed?

- (A) Anya is printed
- (B) Cathy is printed
- (C) Cathy Anya is printed
- (D) Anya Cathy is printed

18. Consider the following code segment.

```
int x = (int) (2*Math.random()) + (int) (2*Math.random());
System.out.println(x);
```

Which of the following could be printed by the code segment?

- (A) 2 only
- (B) 0 and 2 only
- (C) 1 and 2 only
- (D) 0, 1, and 2

19. What is the result when the following code segment is executed?

```
Object[] nums = {null, null, null};
nums[0] = new Integer(2);
nums[1] = new Double(0.718);

System.out.println(nums[0].toString() +
    nums[1].toString() + nums[2].toString());
```

- (A) 20.718 is printed
- (B) 20.718null is printed
- (C) NullPointerException
- (D) ArrayIndexOutOfBoundsException

20. Which of the following statements DOES NOT print 1776?

- (A) `System.out.println("1776");`
- (B) `System.out.println(17 + 76 + "");`
- (C) `System.out.println("" + 17 + 76);`
- (D) `System.out.println("17" + 76);`

21. Consider the following method.

```
private int swap(int a, int b)
{
    if (a < b)
    {
        b = a;
        a = b;
    }
    return b - a;
}
```

What are the values of the variables a, b, and c after the following statements are executed?

```
int a = 2, b = 5;
int c = swap(a, b);
```

- (A) 2, 5, 0
- (B) 2, 5, 3
- (C) 2, 2, 0
- (D) 5, 2, 3

22. Consider the following implementation of Mergesort.

```
public class Mergesort
{
    /**
     * Returns a new array that holds the values
     * arr[m], arr[m+1], ... arr[n] arranged in ascending order.
     * Precondition: 0 <= m <= n < arr.length
     */
    public static int[] sort(int[] arr, int m, int n)
    {
        int[] result = new int[n - m + 1];

        if (m == n)
        {
            result[0] = arr[m];
        }
        else
        {
            int mid = (n + m) / 2;
            int[] result1 = sort(arr, m, mid);
            int[] result2 = sort(arr, mid + 1, n);
            result = merge(result1, result2);
        }

        return result;
    }

    /**
     * Merges arr1 and arr2 so that the resulting array is sorted
     * in ascending order. Returns the resulting array.
     * Precondition: arr1 and arr2 are sorted in ascending order
     */
    private static int[] merge(int[] arr1, int[] arr2)
    { < implementation not shown > }
}
```

If `int[] arr` holds eight values and `Mergesort(arr, 0, 7)` is called, how many times in total will Mergesort's merge method be called?

- (A) 1
- (B) 3
- (C) 7
- (D) 8

23. What is the value of `v[4]` after the following code segment is executed?

```
int d = 1;
int[] v = {1, 1, 1, 1, 1};

for (int i = 0; i < v.length; i++)
{
    d *= 2;
    v[i] += d;
}
```

- (A) 32
- (B) 33
- (C) 64
- (D) 65

24. The statement

```
System.out.println(Integer.MAX_VALUE);
```

prints 2147483647, which is $2^{31} - 1$. What is the result when

```
System.out.println(2*Integer.MAX_VALUE + 2);
```

is compiled/executed?

- (A) 0 is printed
- (B) 4294967296, which is 2^{32} , is printed
- (C) `ArithmeticException`
- (D) Syntax error: cannot apply the + operator to an `Integer` and an `int`

25. Consider the following method.

```
public int mystery(int n)
{
    if (n == 1)
    {
        return 3;
    }
    else
    {
        return 3*mystery(n-1) + 1;
    }
}
```

What does `mystery(4)` return?

- (A) 10
- (B) 31
- (C) 94
- (D) 283

26. Given

```
String s = "ALASKA";
int position = s.substring(1, 3).indexOf("S");
```

what is the value of `position`?

- (A) 1
- (B) 2
- (C) 3
- (D) -1

27. Suppose a class `MyTest` has an instance variable `score`:

```
private int score = 0;
```

Which of the following could serve as a constructor in this class?

- (A) `public MyTest() { }`
- (B) `public int MyTest() { return score; }`
- (C) `public void MyTest(int s) { score = s; }`
- (D) `public MyTest MyTest(int s) { score = s; return this; }`

28. Consider the following method.

```
public boolean examine(String[] letters)
{
    int count = 0;

    for (String ltr1 : letters)
    {
        for (String ltr2 : letters)
        {
            if (ltr1.equals(ltr2))
            {
                count++;
            }
        }
    }

    return count > 0;
}
```

What will the call to examine return for the following arrays?

```
String[] letters1 = {"A", "B", "C"};
String[] letters2 = {"A", "B", "B"};
String[] letters3 = {"A", "A", "B"};
```

	<u>letters1</u>	<u>letters2</u>	<u>letters3</u>
(A)	true	true	true
(B)	true	true	false
(C)	false	true	true
(D)	false	false	true

29. What is printed when the code segment is executed?

```
int[] a = {0, 1};
int[] b = a;
a[0] = 1;
b[0] = 2;
System.out.println(a[0] + b[0] + a[1] + b[1]);
```

- (A) 3
- (B) 4
- (C) 5
- (D) 6

30. Consider the following class.

```
public class Game
{
    private static int bestScore;
    private int score;
    private String player;

    < constructors and methods not shown >
}
```

Which of the following constructors or methods in Game will cause a syntax error?

- (A)
- ```
public Game()
{
 score = 0;
 bestScore = 0;
}
```
- (B)
- ```
public static void resetScore()
{
    score = 0;
    bestScore = 0;
}
```
- (C)
- ```
public void setPlayer(String name)
{
 player = name;
}
```
- (D) None of the above

31. What are the values of a and b after two iterations through the for loop?

```
int a = 1, b = 2;
for (int k = 1; k <= 20; k++)
{
 int oldA = a;
 a = b;
 b = (1 + b) / oldA;
}
```

- (A) 2 and 3  
(B) 3 and 2  
(C) 3 and 3  
(D) 3 and 6

32. Consider the following code segment.

```
ArrayList<String> cities = new ArrayList<String>();
cities.add("Atlanta");
cities.add("Boston");
cities.add("Chicago");

for (String city : cities)
{
 city = city.substring(1);
}

System.out.println(cities);
```

What is printed when the code segment is executed?

- (A) [A, B, C]
- (B) [C, B, A]
- (C) [tlanta, oston, hicago]
- (D) [Atlanta, Boston, Chicago]

33. Consider the following method.

```
public void splat(String s)
{
 if (s.length() < 8)
 {
 splat(s + s);
 }
 System.out.println(s);
}
```

What is printed when `splat("**")` is called?

- (A) \*\*
- (B) \*\*\*\*
- (C) \*\*\*\*  
\*\*
- (D) \*\*\*\*  
\*\*\*\*  
\*\*

34. Suppose a Scanner object `input` is associated with a file open for reading. The file contains integers, one or more per line. If a line contains two or more integers, they are separated by commas. For example:

|          |
|----------|
| 97       |
| 95,87,90 |
| 84       |
| 89,72    |

Which of the following code segments will calculate the sum of all the integers in the file?

- (A)
- ```
int sum = 0;
while (input.hasNext())
{
    sum += input.nextInt();
}
```
- (B)
- ```
int sum = 0;
while (input.hasNext())
{
 sum += Integer.parseInt(input.next());
}
```
- (C)
- ```
int sum = 0;
while (input.hasNext())
{
    String line = input.nextLine();
    String[] nums = line.split(",");
    for (String str : nums)
    {
        sum += Integer.parseInt(str);
    }
}
```
- (D)
- ```
int sum = 0;
while (input.nextLine())
{
 String line = input.nextLine();
 if (line.indexOf(",") == -1)
 {
 sum += Integer.parseInt(line);
 }
 else
 {
 String[] nums = line.split(",");
 for (String str : nums)
 {
 sum += Integer.parseInt(str);
 }
 }
}
```

35. What is printed when the following code segment is executed?

```
int[] factors = {2, 3, 5};
ArrayList<Integer> nums = new ArrayList<Integer>();
nums.add(1);

for (int f : factors)
{
 int n = nums.size();
 for (int k = 0; k < n; k++)
 {
 nums.add(f * nums.get(k));
 }
}

int n = nums.size();
System.out.print(n + " " + nums.get(n-2) +
 " " + nums.get(n-1));
```

- (A) 4 3 5
- (B) 4 6 15
- (C) 8 10 15
- (D) 8 15 30

36. Consider the following method.

```
/**
 * Preconditions: a != null; a.length > 0
 */
private static void doIt(double[] a)
{
 for (int k = 0; k < a.length / 2; k++)
 {
 double temp = a[k];
 a[k] = a[a.length - 1 - k];
 a[a.length - 1 - k] = temp;
 }
}
```

Which of the following best describes the task performed by this method?

- (A) Sorts an array in ascending order
- (B) Swaps the first and last elements of an array
- (C) Reverses the order of elements in an array
- (D) None of the above tasks is implemented correctly

## 37. The method

```
public static boolean testSumOdds(int n)
{
 int sum = 0;

 for (int k = 1; k <= n; k++)
 {
 < missing statement >

 if (sum != k*k)
 return false;
 }
 return true;
}
```

tests the hypothesis that  $1 + 3 + 5 + \dots + (2k - 1) = k^2$  for all positive integers  $k$ . This method returns `true` if the formula holds for  $k$  from 1 to  $n$  and returns `false` if the formula fails for some  $k$  in that range. Which of the following could replace `< missing statement >`?

- (A) `sum += k;`
- (B) `sum += k-1;`
- (C) `sum += 2*k-1;`
- (D) `sum += 2*k+1;`

38. Consider the following incomplete method, which shuffles a list of `Card` objects so that any card can end up at any index in the list `deck` with equal probability.

```
public void shuffle(ArrayList<Card> deck)
{
 int n = deck.size();
 while (n > 1)
 {
 < missing statement >

 Card temp = deck.set(k, deck.get(n-1));
 deck.set(n-1, temp);
 n--;
 }
}
```

Which of the following can replace `< missing statement >` so that the method works as intended?

- (A) `int k = Math.random(n);`
- (B) `int k = Math.random(deck.size() - 1);`
- (C) `int k = (int)(n * Math.random());`
- (D) `int k = (int)((n-1) * Math.random());`

39. Consider the following method sort.

```
public static void sort(int[] a)
{
 for (int i = 1; i < a.length; i++)
 {
 int temp = a[i];
 int j = i-1;

 while (j >= 0 && a[j] > temp)
 {
 a[j+1] = a[j];
 j--;
 }

 a[j+1] = temp;
 }
}
```

Which sorting algorithm is implemented in this method?

- (A) Insertion Sort
- (B) Selection Sort
- (C) Mergesort
- (D) A hybrid algorithm that combines Selection Sort and Insertion Sort

40. Consider the following recursive method.

```
public static int fun(int n)
{
 int product = 2;
 for (int k = 2; k < n; k++)
 {
 product *= fun(k);
 }
 return product;
}
```

What does fun(5) return?

- (A) 24
- (B) 120
- (C) 256
- (D) Nothing. The method exceeds the maximum allowed recursion depth and the program is aborted, because the method does not specify a base case.

41. Consider the following method.

```
public int[][] makeCounts(int n)
{
 int[][] grid = new int[3][n];
 grid[0][0] = 0;
 grid[1][0] = 0;
 grid[2][0] = 1;

 for (int k = 1; k < n; k++)
 {
 grid[0][k] = grid[0][k-1] + grid[1][k-1];
 grid[1][k] = grid[1][k-1] + grid[0][k-1] +
 grid[2][k-1];
 grid[2][k] = grid[2][k-1] + grid[1][k-1];
 }
 return grid;
}
```

What values are in the array returned by `makeCounts(5)`?

(A)

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |

(B)

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 |
| 0 | 1 | 2 | 3 | 4 |
| 1 | 2 | 3 | 4 | 5 |

(C)

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 3 | 6 |
| 0 | 1 | 2 | 5 | 8 |
| 1 | 1 | 4 | 7 | 9 |

(D)

|   |   |   |   |    |
|---|---|---|---|----|
| 0 | 0 | 1 | 3 | 8  |
| 0 | 1 | 2 | 5 | 12 |
| 1 | 1 | 2 | 4 | 9  |

42. Consider the following class `Athlete`.

```
public class Athlete
{
 private int numMedals;

 public int getRank() { return numMedals; }

 public int compareTo(Athlete other)
 {
 // return numMedals - other.numMedals;
 return getRank() - other.getRank();
 }

 < constructors and other methods not shown >
}
```

As you can see, the programmer has commented out direct references to `Athlete`'s instance variable `numMedals` in the `compareTo` method and replaced them with calls to the `getRank` method. What is the most compelling reason for doing this?

- (A) To correct a syntax error: being private, neither `numMedals` nor `other.numMedals` is directly accessible in the method's code
- (B) To correct a syntax error: being private, `other.numMedals` is not directly accessible in the method's code (`numMedals` is replaced with `getRank()` for consistency)
- (C) To avoid possible problems later: if `other` happens to be an object of a subclass of `Athlete` in which `numMedals` is not used in calculating the rank, the original code would fail
- (D) To achieve better encapsulation of the `Athlete` class



## Practice Exam #2

### SECTION II

Time — 90 minutes

Number of questions — 4

Percent of total grade — 45

1. A Grammy is awarded to recognize outstanding achievement in music. An Oscar is awarded for artistic and technical merit in film. The Tony award recognizes excellence in live Broadway theater. The Emmy award is presented for artistic and technical merit in the television industry. The Grammy was first awarded in 1959; the Oscar in 1929; the Tony in 1947; and the Emmy in 1949.

Several outstanding artists have achieved “EGOT” status by winning all four of these awards. The Java class `EgotStatus` makes a list of all the awards (represented by a `String`) received by a particular artist and determines whether this artist has reached EGOT status, and if so, in what year.

A partial definition of the `EgotStatus` class is shown below. You will write two methods of this class.

```
public class EgotStatus
{
 /** The string that holds the list of all awards received
 by the artist */
 private String allAwards = "";

 /**
 * Returns the current year
 */
 public int currentYear()
 { /* implementation not shown */ }

 /**
 * If the artist received one of the four major awards
 * in the specified year, returns the award name ("Emmy",
 * "Grammy", "Oscar", or "Tony"); otherwise returns null.
 */
 public String getAward(int year)
 { /* implementation not shown */ }
```

*Continued*



```

/**
 * Makes a string of all received awards and saves it in
 * the instance variable allAwards in the format
 * "yyyy award, yyyy award, ..., yyyy award",
 * where yyyy stands for the year of the award and
 * award is "Emmy", "Grammy", "Oscar", or "Tony".
 */
public void makeAwards()
{ /* to be implemented in Part (a) */ }

/**
 * Returns "Reached EGOT status in yyyy" if this artist
 * first reached EGOT status in yyyy, where yyyy is
 * the actual year of the award; otherwise returns
 * "yyyy Not EGOT yet", where yyyy is the current year.
 * Postcondition: allAwards is unchanged
 */
public String reachedEgot()
{ /* to be implemented in Part (b) */ }
}

```

- (a) Write the method `makeAwards` that sets the instance variable `allAwards` to the string representation of all awards received by this artist. For each received award, the string "yyyy award" is added to `allAwards`. The individual award substrings are separated by ", ". For example, at the end of 2024, the value of the instance variable `allAwards` for Whoopi Goldberg would be:

"1986 Grammy, 1990 Oscar, 2002 Emmy, 2002 Tony, 2009 Emmy"

This string is compiled by calling the `getAward` method for every year from 1950 to the current year. Notice that the same artist can receive two or more awards in the same year.

Complete the `makeAwards` method below. Your solution should not use arrays, ArrayLists, or any other data structures. The only method of the `String` class you may use in your solution to Part (a) is `length`.

```

/**
 * Makes a string of all received awards and saves it in
 * the instance variable allAwards in the format
 * "yyyy award, yyyy award, ..., yyyy award",
 * where yyyy stands for the year of the award and
 * award is "Emmy", "Grammy", "Oscar", or "Tony".
 */
public void makeAwards()

```

- (b) Write the method `reachedEgot` that analyzes the `allAwards` string and determines the first year this artist reached EGOT status. `reachedEgot` returns "Reached EGOT status in yyyy" if this artist first achieved EGOT status in yyyy. For example, for Whoopi Goldberg, whose `allAwards` string is

"1986 Grammy, 1990 Oscar, 2002 Emmy, 2002 Tony, 2009 Emmy"

`reachedEgot` will return "Reached EGOT status in 2002". If EGOT status has not been achieved, `reachedEgot` returns "yyyy Not EGOT yet", where yyyy is the current year. For example, "2026 Not EGOT yet".

Complete the `reachedEgot` method below.

```
/**
 * Returns "Reached EGOT status in yyyy" if this artist
 * first reached EGOT status in yyyy, where yyyy is
 * the actual year of the award; otherwise returns
 * "yyyy Not EGOT yet", where yyyy is the current year.
 * Postcondition: allAwards is unchanged
 */
public String reachedEgot()
```

❖ ❖ ❖

Class information for this question

```
public class EgotStatus
private String allAwards

public int currentYear()
public String getAward(int year)
public void makeAwards()
public String reachedEgot()
```

¶ For additional practice, complete the `EgotStatus` class.

¶ Write a `TestEgot` class with a main method to test your `EgotStatus` class.

2. The class `Shopping` helps a consumer choose the best option for buying a particular item. `Shopping`'s constructor initializes an `int` constant that represents the maximum distance (in miles) the consumer is willing to travel to a store.

The `Shopping` class contains an `addShoppingOption` method, which takes three parameters.

- The first parameter is a `String` that represents the store name.
- The second parameter is a `double` that represents the price at that store.
- The third parameter is an `int` that represents the distance to that store.

If the distance to the store doesn't exceed the maximum allowed distance, `addShoppingOption` updates the best store and the best price. This method returns a `String` that includes the name of the best store found so far and the best price at that store.

The following table contains a sample code execution sequence and the corresponding return values of the `addShoppingOption` method. The code execution sequence appears in a class other than `Shopping`.

| Statement                                                         | Value Returned<br>(blank if no value) | Explanation                                                                                       |
|-------------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>Shopping kayak = new<br/>Shopping(20);</code>               |                                       | <code>Shopping</code> object <code>kayak</code> is created with the maximum distance of 20 miles. |
| <code>kayak.addShoppingOption(<br/>"JMart", 225.95, 18)</code>    | JMart price 225.95                    | Best price set to 225.95.                                                                         |
| <code>kayak.addShoppingOption(<br/>"Jest Buy", 189.00, 24)</code> | JMart price 225.95                    | Best price remains 225.95 because the store is too far.                                           |
| <code>kayak.addShoppingOption(<br/>"Tarjet", 220.95, 19)</code>   | Tarjet price 220.95                   | Best price and store name are updated.                                                            |
| <code>kayak.addShoppingOption(<br/>"Reij", 222.50, 15)</code>     | Tarjet price 220.95                   | Best price remains the same because Reij price is higher than what had been found earlier.        |

Write a complete `Shopping` class. Your implementation must meet the specifications, and the format of the output must conform to the example shown in the above table. Do not use arrays, `ArrayLists`, or other data structures in your solution.

- For additional practice, allow the consumer to order the item online, using 0 distance in `addShoppingOption` calls. Add a method that sets the price of gas and the car's miles per gallon attribute and adds to the price the cost of gas for travel to a "brick-and-mortar" (not online) store and back.

3. Apflix recommends movies to customers based on their previous selections. Clearly, some movie-matching mechanism is employed. A movie is described by a set of predefined features, such as genre, rating, director, lead actress, lead actor, and so on. A movie is represented by an object of the class `Movie`. Its method

```
public double likenessScore(Movie other)
```

returns the likeness score between this movie and other.

For each subscriber, Apflix keeps a list of movies they have watched recently. Some of the movies on that list may not fit well with the rest (for example, a movie may have been watched by a family member or friend). Apflix's marketing department wishes to detect and remove such "outliers" from each subscriber's movie list.

An Apflix subscriber is represented by the class `ApflixSubscriber`. A partial definition of this class is shown below.

```
public class ApflixSubscriber
{
 /** The list of movies watched by this subscriber */
 private ArrayList<Movie> movies;

 /**
 * Initializes the instance variable movies and copies
 * all the data from mvs to movies.
 */
 public ApflixSubscriber(ArrayList<Movie> mvs)
 /* implementation not shown */

 /**
 * Returns a list of values that measure how well each
 * movie in the movies list fits with the rest
 * of the movies in the list. The value of the k-th
 * element is the average of the likeness
 * scores between the k-th movie and the rest of the
 * movies in the list.
 * Precondition: The size of the list movies is
 * not less than 2
 * Postcondition: the list movies remains unchanged
 */
 private ArrayList<Double> getFitCoefficients()
 { /* implementation not shown */ }
```

*Continued*



```

/**
 * Obtains the fit coefficients for the movies list,
 * calculates their average, and removes
 * from the movies list all the elements whose fit
 * coefficient is less than half of the average.
 */
public void removeOutliers()
{ /* to be implemented */ }

/* There may be instance variables, constructors, and
 methods that are not shown. */
}

```

The method `getFitCoefficients` returns a list of `Double` values that measure how well each movie in the list `movies` fits with the rest. The size of the list returned by `getFitCoefficients` is the same as the size of the `movies` list. The  $k$ -th element in the returned list is equal to the fit coefficient for the  $k$ -th movie.

Write the method `removeOutliers`. This method obtains a list of the fit coefficients for the `movies` list, calculates their average, and uses half of that value as a threshold. It then removes from `movies` all the elements whose fit coefficient is less than the threshold.

Complete the method `removeOutliers` below.

```

/**
 * Obtains the fit coefficients for the movies list,
 * calculates their average, and removes
 * from the movies list all the elements whose fit
 * coefficient is less than half of the average.
 */
public void removeOutliers()

```



Class information for this question

```

public class ApflixSubscriber
private ArrayList<Movie> movies

public ApflixSubscriber(ArrayList<Movie> mvs)
private ArrayList<Double> getFitCoefficients()
public void removeOutliers()

```

¶ For additional practice, write the `Movie` class. Represent the features of a movie by a string of length  $3*n$  — the same length for all movies — and add a constructor to set that string. Each feature is a substring of three characters. The likeness score is the number of matched features divided by  $n$ .

Implement the `getFitCoefficients` method of the `ApflixSubscriber` class. The fit coefficient for a movie is defined as the average of the likeness scores between this movie and all the other movies in the list. Make sure that the likeness score for any pair of movies from the list is computed only once, but do not use any temporary arrays or other data structures.

¶ Add a main method to the `ApflixSubscriber` class and test it.

4. The *Sliding Tiles* game consists of a rectangular grid with marked square cells. A cell can either be empty or hold a tile with a letter on it. Figure 1 shows an example.

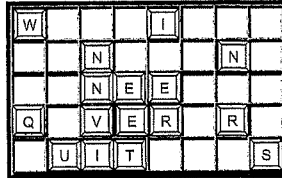


Figure 1

A player can slide a tile to a neighboring empty cell — up, down, left, or right. Let us call a grid of tiles “packed” if no tile has a neighboring empty cell above or to the left of it. Figure 2 shows an example of a packed grid.

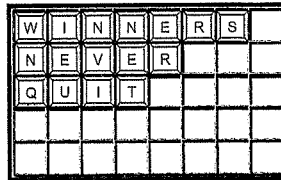


Figure 2

A computer model of the game is represented by the class `TilesGame`. A partial definition of this class is shown below. You will write one method of this class.

```
public class TilesGame
{
 /** the grid of tiles */
 private String[][] tiles;

 /**
 * Constructs a TilesGame object from a 2D array
 * of letters.
 */
 public TilesGame(String[][] letters)
 { /* implementation not shown */ }
```

Continued



```
/**
 * Returns true if any element of tiles not equal to null
 * does not have a null in the same column in the
 * previous row or in the previous column in the same row;
 * otherwise returns false.
 * Precondition: tiles has at least one row and one column
 * and is not empty, that is, at least one
 * element is not null
 */
public boolean isPacked()
{ /* to be implemented */ }

/* There may be instance variables, constructors, and methods
 that are not shown. */
}
```

The tiles instance variable of this class is the grid of cells. A tile is represented by a one-letter literal String object; an empty cell holds a null.

Complete the isPacked method below.

```
/**
 * Returns true if any element of tiles not equal to null
 * does not have a null in the same column in the
 * previous row or in the previous column in the same row;
 * otherwise returns false.
 * Precondition: tiles has at least one row and one column
 * and is not empty, that is, at least one
 * element is not null
 */
public boolean isPacked()
```

❖ ❖ ❖

Class information for this question

```
public class TilesGame
private String[][] tiles
public TilesGame(String[][] letters)
public boolean isPacked()
```

¶ For additional practice, implement the constructor and additional methods of the `TilesGame` class:

```
/**
 * Relocates the tile at position (row, col) by first
 * "sliding" it up to the adjacent earlier row
 * in the same column 0 or more times, as long as the
 * cell there is empty. When the vertical "slide" is
 * finished, slides the tile to the left to the adjacent
 * column in the same row 0 or more times, as long as the
 * cell there is empty.
 * Preconditions: the 2D array tiles is not empty;
 * the cell at location (row, col) holds a tile.
 */
private void moveTile(int row, int col)
{ /* implementation not shown */ }

/**
 * Packs the grid by traversing the tiles and moving each tile
 * as described in the moveTile method.
 */
public void pack()
{ /* implementation not shown */ }
```

To pack a given configuration of tiles, scan the whole grid row by row, starting at the top row, scanning each row from left to right. When you encounter a tile, move the tile by sliding it up as many times as possible, then to the left as many times as possible. Write a private helper method `moveTile` that helps to implement the packing algorithm.

Write a static void method `display(String[][] tiles)` that prints out the grid, replacing each `null` with a dot. Declare a grid that matches Figure 1 above, print it out, pack it, and print out the resulting grid. The output should match the grid in Figure 2.

To test your code further, repeat the following 100 times: create an 8-by-5 grid filled with random letters and several `null`s in random places, then pack the grid and verify that it is indeed packed.